



Universidad de Jaén

Tema 2.2 Desarrollo web en Jakarta EE

Arquitectura MVC, plantillas JSP y JSTL

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

<http://tiny.cc/dawuja>

Este obra está bajo una Licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional.

v 2.3

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Objetivos

- Comprender el funcionamiento de la arquitectura **MVC de una aplicación web**
- Conocer y saber utilizar mecanismos en la arquitectura JEE para **implementar el modelo MVC en una aplicación web**
- Comprender las ventajas del **uso de plantillas** para generación de contenidos dinámicos y conocer sus tipos
- Conocer y saber utilizar los **contextos de una aplicación web** como mecanismo para intercambio de información entre Servlets y páginas JSP
- Conocer y saber usar **etiquetas JSP y Expression Language** para simplificar la generación de contenidos en una página JSP
- Conocer la **biblioteca de etiquetas JSTL** y saber utilizarla en páginas JSP

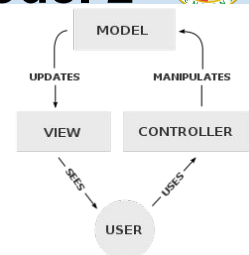


Organizando el flujo de ejecución de una aplicación web

Arquitectura Modelo Vista Controlador

El modelo MVC en aplicación web: Model 2

- La naturaleza del protocolo HTTP obliga a hacer ciertas adaptaciones sobre el patrón MVC original ¿pueden cambios en modelo refrescar la vista en el navegador?



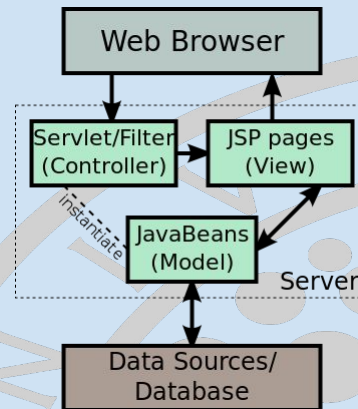
Fuente: Wikipedia
<https://es.wikipedia.org/wiki/Modelo-vista-controlador>

- **Model 2** es la adaptación del patrón MVC a la tecnología JavaServer Pages (Govind Seshadri, 1999)
 - Model 1, sería una APP web implementada íntegramente sobre Servlets/JSP que realizan indistintamente labores de Modelo, Vista y/o Controlador
- **Elementos constituyentes MVC en aplicación web (model 2):**
 - Los Servlets realizan la labor del **controlador**
 - Las JSP se encargan de la capa de presentación (**vista**)
 - Los Beans y/o POJOs (*Plain Old Java Objects*) estarían asociados al **modelo** de la aplicación

Diagrama MVC en aplicación web

Fases petición HTTP

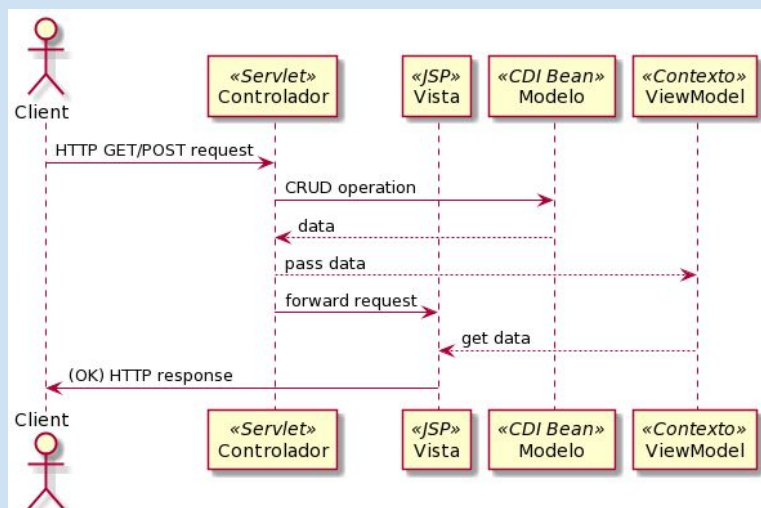
1. El **servlet** recibe la petición http
2. Valida datos
3. Intercambia datos con el **modelo**
4. Instancia **Beans/POJOs** que coloca en diferentes **contextos**
5. **Transfiere el control** a la vista (JSP) adecuada
6. La página **JSP** recupera la información del contexto adecuado y genera la respuesta



Fuente: http://en.wikipedia.org/wiki/Model_2

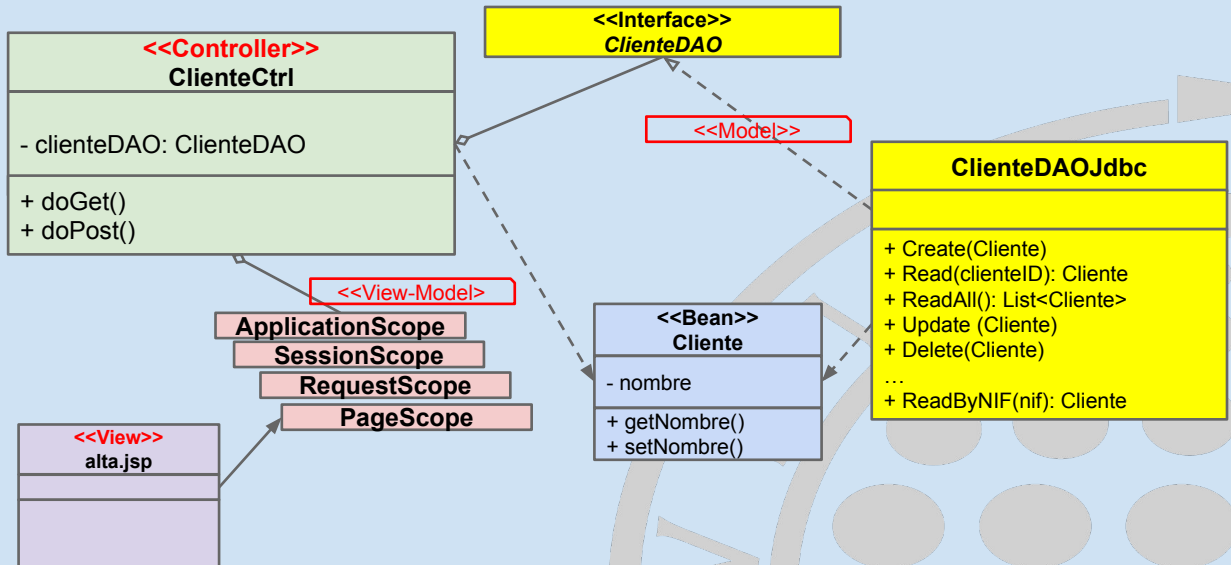
5

Diagrama de secuencia MVC Model 2



6

Arquitectura MVC aplicación web



7

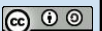
Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Plantillas de contenidos HTML dinámicos

EL/JSTL

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Uso de plantillas en HTML

- **Problema**

- Los scriptlets mezclan lógica de control (java) y presentación (html)

- **Solución**

- **Evitar el uso de código en documentos html**
- *Utilizar elementos textuales o **plantillas** que serán reemplazados en el servidor por contenidos generados dinámicamente*
- Uso de un **motor de renderizado (template engine)**
 - *Misión:* localizar patrones de texto con un formato específico para sustituirlo por datos del programa
 - Sin el motor, los elementos son texto plano
- **Los Servlets proporcionarán los datos** a los motores de renderizado



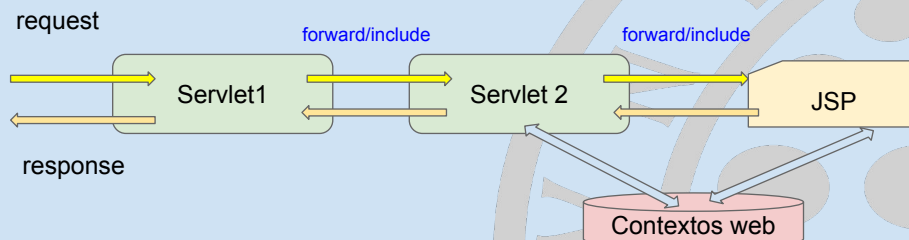
Motores de plantillas JVM

- La especificación **JSP** permite crear etiquetas similares a html que encapsulan código html/css/js y java
 - Se identifican usando un prefijo (espacio de nombres)
 - Etiquetas predefinidas JSP (prefijo jsp):
 - e.g. `<jsp:include page="/WEB-INF/jsp/navegacion.jsp" >`
 - Biblioteca de etiquetas JSP personalizadas o estándar (JSTL)
 - e.g. `<app:ficha-cliente datos="{cliente}/>`, `<c:import url="/ofertas.jsp" />`
- **Expression Language (EL)**
 - para acceso a objetos, métodos y uso de expresiones.
 - `<p> La cuota con IVA de ${nombre} es ${cuota*1.28} </p>`
- **Otros motores de plantillas JVM**
 - Trabajan directamente sobre ficheros: html, xml... (no JSP)
 - Ejemplos: **Facelets**, **Tiles**, **FreeMarker**, **Velocity**, **Thymeleaf**, **Moustache**,

Intercambio de información entre Controladores y Vistas

Reenvío de peticiones entre servlets

- Un servlet puede **delegar todo o parte del procesamiento** de una petición a otro servlet/jsp
 - Reenvío parcial (**include**): el servlet destino genera *parte* de la salida
 - Equivalente a `<jsp:include page="recurso"/>`
 - Reenvío completo (**forward**), el servlet destinatario gestionará la salida *completa*
 - Equivalente a `<jsp:forward page="recurso"/>`
- Antes de transferir el control de la petición puede enviar información adicional al servlet o jsp destinatario utilizando **contextos web**



Delegación de peticiones HTTP

- Se utiliza un objeto **RequestDispatcher** para transferir la petición (request) en curso

```
RequestDispatcher rd=request.getRequestDispatcher("/url-servlet-jsp");
```

- Tipos de delegación:**

- **Parcial:** `rd.include(request,response);`

- El recurso genera *parte* de la respuesta. e.g. una cabecera o pie de página
- equivalente a `<jsp:include page="/url">` en JSP con url relativas
- equivalente a `<c:import url="/url">` en JSTL con url genéricas

- **Total:** `rd.forward(request,response);`

- La petición se transfiere *totalmente* al nuevo recurso
- equivalente a `<jsp:forward page="/url">` en JSP

- Los ficheros JSP con **plantillas suelen estar ocultos**, e.g. /WEB-INF/jsp/encabezado.jsp

- Ejemplo:**

```
private void doGet (request, response){  
    /* procesar petición y colocar datos para el JSP en algún contexto */  
    request.getRequestDispatcher("/WEB-INF/jsp/aviso.jsp")  
        .forward(request,response);  
}
```

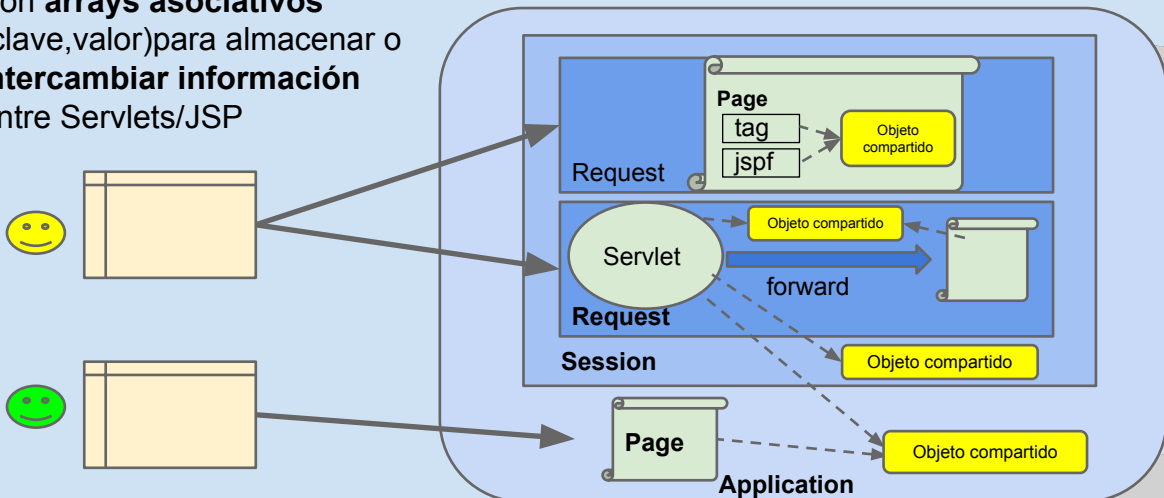
13

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



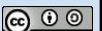
Contextos en una aplicación web

Son **arrays asociativos**
(clave,valor)para almacenar o
intercambiar información
entre Servlets/JSP



14

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



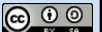
Envío de información a un contexto

- Un servlet debe acceder a cada tipo de contexto a través de objetos específicos que lo gestionan:
- Los contextos son **arrays asociativos** (clave,valor)
 - **request**, asociado a todas las páginas/servlets que intervienen en un única petición
 - `request.setAttribute("clave",valor);`
 - **session**, todas las peticiones realizadas desde un cliente específico y por un tiempo determinado
 - `request.getSession().setAttribute("clave",valor);`
 - **application**, todas las páginas/peticiones de la aplicación
 - `this.getServletContext().setAttribute("clave",valor);`
- Ejemplo

```
private void doGet (request, response){  
    // procesar petición...  
    Usuario usuario = servicioUsuarios.getUsuario();  
    request.setAttribute("usuario",usuario); //enviar datos al contexto request  
    request.getRequestDispatcher("/WEB-INF/jsp/muestra_usuario.jsp")  
        .forward(request,response); //pasar control fichero JSP  
}
```

15 }

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Uso de JSP como plantillas web

Expression Language, Beans y JSTL



Java Unified Expression Language (EL)

- Notación para la inclusión de expresiones en páginas web
- Se utiliza para acceder a información de la aplicación web (localizada en contextos). e.g. `requestScope`
 - variables o expresiones: `${variable}` `${cantidad*1.28}`
 - índices, vectores asociativos/maps, métodos o propiedades de objetos
 - `${ciudades[1]}` `${vector["clave"]}`
 - `${usuario.getNombre()}`
 - `${objeto.propiedad}`
 - `${objeto["propiedad"]}`
 - NO sirve para acceder a variables de script `<%...%>`. Usar `<%=variable%>` en su lugar
- Si el **atributo no existe o devuelve null**, equivale a cadena vacía (no genera errores)
- **Conversión de tipos implícita**: numérico → cadena

17

Ejemplo de JSP usando EL

```
<!DOCTYPE html>
<html lang="ES">
<head>
<title>Ejemplo</title>
<meta charset="UTF-8">
</head>
<body>
  <h1>Datos del usuario</h1>
  <div>Nombre: ${param.nombre}</div>
  <div>Apellidos: ${param["apellidos"]}</div>
  <div>Es socio? ${param.esSocio?"Sí":"No"}</div>
  <footer>Conexión desde ${pageContext.request.remoteAddr}
  <p>Tu navegador es ${header["user-agent"]}</p></footer>
</body>
</html>
```

Recordatorio de misma operación con scriptlets

```
<%= request.getParameter("nombre")%>
```

18

Objetos (*beans*) implícitos en EL

Tipo	Descripción
<code>pageContext</code>	Acceso a toda la información de contexto de la página <code>Inicio</code>
<code>param</code>	Array asociativo con valor de cada parámetro de la petición. Primer valor si hay varios con el mismo nombre
<code>paramValues</code>	Array asociativo con un array de valores para cada parámetro
<code>header</code>	Array asociativo de encabezados http y valores asociados
<code>headerValues</code>	Similar a <code>paramValues</code> pero con valores de encabezados
<code>cookie</code>	Array asociativo con nombres y valores de cookies
<code>initParam</code>	Array asociativo con parámetros de inicialización del servlet (e.g. en <code>web.xml</code>)
<code>pageScope</code> , <code>requestScope</code> , <code>sessionScope</code> , <code>applicationScope</code>	Array asociativo de nombres de variable y sus valores en diferentes contextos de ejecución de la aplicación: por página, por petición http, por sesión de un usuario, para todos los servlets de la aplicación. Se utilizan para persistencia de valores entre llamadas y para intercambio de información.
<code>\${variable}</code>	Acceso a valores almacenados en los contextos anteriores por orden (page→ application)

19

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



JavaBeans

- **Definición original:** *es un componente software reutilizable que puede ser manipulado visualmente desde un IDE*
- **Utilidad en Apps Web**
 - Intercambio de información entre partes de la aplicación utilizando los diferentes contextos
 - Encapsular información del dominio del problema (*modelo*)
 - Creación y **acceso a objetos sin utilizar scriptlets** `<%%>`
 - Acceso a propiedades usando EL: `${bean.propiedad}`
 - Asociación entre propiedades bean y parámetros en formulario
- **Implementación:** Es una clase normal (POJO) con ciertos requisitos:
 - Constructor por *defecto*
 - métodos *set*, *get/is*, para sus atributos (NO públicos)
 - Implementar la interfaz *java.io.Serializable* en ciertas situaciones
 - Ubicados en paquetes

20

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Propiedades en beans

Ejemplo bean:



- El **estado** (atributos) de un bean se representa mediante propiedades
- El **identificador** de las propiedades se definen a partir de métodos tipo get/is y set:

- Las propiedades que pueden **leerse** deben definir un método *getPropiedad* (o *isPropiedad* para boolean)
- Las propiedades que pueden **modificarse** definen métodos *setPropiedad*
- El **identificador** comienza por minúscula (o todo mayúsculas si dos primeras letras en mayúsculas)

■ Ejemplos:

getNombre → **nombre**, *isSocio* → **socio**, *getDirCorreo* → **dirCorreo**, *getDNI* → **DNI**

```
package es.ujaen.daw.club.modelo.Cliente;
class Cliente implements Serializable {
    private String nomCliente;
    private boolean esSocio;
    public Cliente() {
        nomCliente="";esSocio=false;}
    public String getNombre() {
        return nomCliente;}
    public void setNombre(String nom) {
        nomCliente=nom;}
    public boolean isSocio() {
        return esSocio;}
    public void setSocio(boolean socio) {
        esSocio=socio;}
}
```

21

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Ejemplo de formulario para mostrar datos un usuario



```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<jsp:useBean id="cliente" type="es.ujaen.daw.club.modelo.Cliente" scope="request" />
<!DOCTYPE html>
<html>
    <head> <title>Alta Cliente</title> </head>
    <body >
        <h1>Datos del usuario</h1>
        <section >
            <div><span class="etiqueta">Nombre</span>${cliente.nombre}</div>
            <div><span class="etiqueta">Apellidos</span>${cliente.apellidos}</div>
            <div><span class="etiqueta">Socio</span>${cliente.socio?'Sí':'No'}</div>
        </section>
    </body>
</html>
```

Opcional <jsp:useBean/>

- Permite que el IDE conozca el tipo de la variable cliente en EL y pueda autocompletar propiedades o métodos del mismo
- Indica el contexto inicial donde localizar el bean (o se buscará secuencialmente en todos)

22

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Resumen de acceso a contextos

- Son **arrays asociativos** (clave,valor) para almacenar o intercambiar información entre Servlets/JSP
 - **page**, asociado a una petición HTTP sobre un JSP. Se elimina al salir de la página
 - `pageContext.set/getAttribute("clave",valor);` //en JSP
 - `${pageScope.clave}` //Usando EL
 - **request**, asociado a todas las páginas/servlets que intervienen en una única petición
 - `request.set/getAttribute("clave",valor);` o `${requestScope.clave}`
 - **session**, todas las peticiones realizadas desde un cliente específico y por un tiempo determinado
 - `session.set/getAttribute("clave",valor);` o `${sessionScope.clave}`
 - **application**, todas las páginas/peticiones de la aplicación
 - `getServletContext()`, objeto *application* o `${applicationScope.clave}`
- En EL, **si no se especifica ámbito** de la clave, se busca del primero (*page*) al último (*application*)

23

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Java Standard Tag Library

JSTL

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Etiquetas personalizables JSP

- JSP permite definir **etiquetas (Tag) personalizadas** para reutilizar contenidos y/o facilitar ciertas tareas habituales
- Las etiquetas permiten: **evitar código java** junto al html, **modularidad**, **reutilización**
- **Sintaxis**
 - Similar a las etiquetas html o xml
 - Utilizan un prefijo (espacio de nombres) para diferenciarlas de html o xml
 - dobles `<prefijo:tag attr1="val1" ... > </prefijo:tag>`
 - simples `<prefijo:tag attr1="val1" ... />`
- Se organizan en bibliotecas
- **Directiva** `<%@taglib uri="id_biblioteca" prefix="prefijo"%>`
 - Indica en el JSP dónde localizar las nuevas etiquetas y el prefijo que las identificará

`<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>`

`<%@taglib uri="jakarta.tags.core" prefix="c"%> <!-- id en JakartaEE 10-->`

25

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024

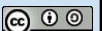


Etiquetas personalizadas básicas

- Permiten agrupar elementos de vista reutilizables
- **Características básicas**
 - Permiten integrar contenido JSP e incluso anidar otras etiquetas
 - Extensión `.tag`
 - Definidas mediante directiva `<%@tag %>` (similar a `@page`)
 - Atributos declarados con `<%@attribute name="attr1" required="true"%>`
 - Usos avanzados: fragmentos e implementación en Java [+info](#)
- **Utilización**
 - Se agrupan en bibliotecas: `/WEB-INF/tag` o en `.jar` independientes
 - Acceso mediante espacios de nombres:
`<%@taglib prefix="club" tagdir="/WEB-INF/tags/club" %><!-- /WEB-INF/tags/club/footer.tag-->`
`<club:footer message="Club de Tennis" />`

26

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Ejemplo de etiqueta personalizada

```
<!-- /WEB-INF/tags/club/footer.tag -->
<%@tag description="Basic footer" pageEncoding="UTF-8"
import="java.time.LocalDate"%>
<%@attribute name="message" required="true"%>

<footer class="card bg-light">
  <div class="card-body" >
    <p class="card-text text-right mb-0">${message}</p>
    <p class="card-text text-right text-muted">%=LocalDate.now()%</p>
  </div>
</footer>
```

```
<%@taglib prefix="club" tagdir="/WEB-INF/tags/club" %>

<club:footer message="Club de Tennis" />
```



Club de Tennis
2020-06-08

27

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



JSP Standard Tag Library, JSTL

- Recopilación de etiquetas de uso habitual
- Módulo integrado en especificación JakartaEE10
- **Añadir JSTL a un proyecto**
 - Añadir la biblioteca (jar) JSTL al proyecto
 - No necesario en servidores de apps >=J5EE (e.g. Glassfish). En Tomcat hay que añadirla expresamente
 - Especificar en cada página qué paquete de etiquetas se va a utilizar con la directiva:


```
<%@taglib prefix="pref_etiqueta" uri="identificador" %>
```
 - Por convención se utilizan prefijos habituales para ciertas bibliotecas, evitando conflictos entre unas y otras
 - **c**, biblioteca básica; **fmt**, formato/internacionalización (i18n), ...
- **Ejemplos de selección de módulos JSTL:**

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
```

En IntelliJ añadir la siguiente dependencia al proyecto maven (pom.xml) para que funcione el autocompletado de etiquetas:

```
<dependency>
<groupId>org.glassfish.web</groupId>
<artifactId>jakarta.servlet.jsp.jstl</artifactId>
<version>3.0.0</version>
<scope>provided</scope>
</dependency>
```

Se mantiene compatibilidad con antigua URI
e.g. <http://java.sun.com/jsp/jstl/core>

28

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Módulos JSTL

- **Core** (prefijo: **c**, uri: [jakarta.tags.core](#))
 - Gestión de variables en contextos
 - Control de flujo: condicionales, ciclos, etc.
 - Gestión de URLs
 - Varios
- **Formato** (prefijo: **fmt**, uri: [jakarta.tags.fmt](#))
 - Formato de fechas y números
 - Tratamiento de traducciones de textos (locale)
 - Formato de mensajes
- **Functions** (prefijo: **fn**, uri: [jakarta.tags.functions](#))
 - Tamaño de colecciones
 - Manipulación de cadenas
- **Database** (prefijo: **sql**, uri: [jakarta.tags.sql](#))
 - Conexión JDBC y manipulación de RecordSet. Es preferible no incluir aspectos relacionados con la conexión a la BD en ficheros JSP
- **XML** (prefijo: **x**, uri: [jakarta.tags.xml](#))

29

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Utilización de variables en etiquetas

- **Tipos de definición**
 - Variables definidas **expresamente en una etiqueta**,
 - e.g. `<c:set var="variable" value="valor" scope="..." />`
 - Variables **en contextos**. Acceso usando EL
 - e.g. `${requestScope.variable}` o `${variable}`
 - *Ojo: las variables en scriptlets no son "visibles" y deben pasarse a través de un contexto*
- **Acceso**
 - Referencia a **variables en atributos** de etiquetas.
 - e.g. `<c:out value="variable" escapeXml="true" default="valor si no definida" />`
 - **Ojo:** el atributo opcional `escapeXml` evita inyección de código html/javascript
 - Usando **EL**: e.g. `${variable}` //OJO, no escapa código html

30

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Etiquetas básicas: condiciones

- Simples:

```
<c:if test="${condicion}">
    Contenido
</c:if>
```

Usadas principalmente
para visualización
condicional de contenidos

- Múltiples:

```
<c:choose>
    <c:when test="condicion1">Contenido1</c:when>
    <c:when test="condicion2">Contenido2</c:when>
    ...
    <c:otherwise>Contenido defecto</c:otherwise>
</c:choose>
```

... pero hay que procurar que la lógica de control de la aplicación NO resida en la vista (jsp)

31

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Etiquetas básicas: ciclos

- Iteración sobre valores de variables numéricas

```
<c:forEach var="variable" begin="ini" end="fin" step="salto">
    Contenido ${variable} o <c:out value="variable"/>
</c:forEach>
```

- Iteración sobre conjuntos de elementos:

- arrays, colecciones (List, Map, Set, etc.), Iteradores o enumeraciones, cadena con valores separados por comas

```
<c:forEach var="var" items="${colección}" varStatus="estado">
    ${estado.count} Contenido ${var} o ${var.key} ${var.value} en maps
</c:forEach>
```

- **estado**, objeto con propiedades: *first*, *last* (boolean); *count*; *index* (del elemento en la colección)

32

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Ejemplo: listar parámetros HTTP (con JSTL)

Acceso: <http://localhost:8080/appweb/testparams.jsp?altura=185&socio=on&nombre=María>

```
<%@page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@taglib prefix="c" uri="jakarta.tags.core" %>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type"
content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h2>Parámetros de petición HTTP</h2>
<c:if test="${pageContext.request.getMethod().equals('GET')}">
<strong>Usando método GET </strong>
</c:if>
<ul>
<c:forEach var="par" items="${param}" varStatus="s">
<li>${s.count} <b>${par.key}</b>: <b>${par.value}</b> </li>
</c:forEach>
</ul>
<form action="" method="GET">
<input type="text" name="nombre">
<input type="number" name="altura">
<input type="checkbox" name="socio" >
<input type="submit" value="Enviar">
</form>
</body>
</html>
```

Parámetros de petición HTTP

Usando método GET

- 1 altura: 185
- 2 socio: on
- 3 nombre: María

33

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Ejemplo: listar encabezados HTTP (con JSTL)

```
<%@taglib uri="jakarta.tags.core" prefix="c" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Cabeceras petición HTTP</h1>
<c:if test="${pageContext.request.getMethod().equals('GET')}">
<strong>Usando método GET </strong>
</c:if>
<ul>
<c:forEach var="encab" items="${headerValues}"
varStatus="estado">
<c:forEach var="valor" items="${encab.value}">
<li><b>${encab.key}</b>: <b>${valor}</b> </li>
</c:forEach>
</c:forEach>
</ul>
</body>
</html>
```

Cabeceras petición HTTP

Usando método GET

- 1.- cookie:
JSESSIONID=D4943FCC165487D9F4B7E0EFD0798A6;
JSESSIONID=685b94c59782e2a12a61d9378e6d;
treeForm_tree-hi=treeForm:tree:updateCenterPlugIN
- 2.- cache-control: no-cache
- 3.- connection: keep-alive
- 4.- accept-language: es,en;q=0.8
- 5.- host: localhost:8080
- 6.- accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp
- 7.- user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.107 Safari/537.36
- 8.- accept-encoding: gzip,deflate,sdch
- 9.- dnt: 1
- 10.- pragma: no-cache

34

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Etiquetas básicas: manipulación URL

● Importar un recurso (externo o interno)

- `<c:import url="direccion_web"/>`
- `<jsp:include ...>` sólo sirve para recursos de la propia aplicación web
- se puede almacenar el resultado con los parámetros opcionales `var="variable" scope="contexto"`

● Redirección web

- `<c:redirect url="dir_web"/>`

● Generar URL absoluta

- Añade identificador de sesión (JSESSIONID) a la url si el navegador no acepta cookies y la url es relativa (de la appweb)
- Equivalente a `response.encodeUrl("dir_web")` **Útil para añadir recursos a la página: css, png, js,... sin depender de la URL de la aplicación o del servlet que la utiliza**
- URL relativa a dirección de la aplicación
- `<c:url value="dir_web"/>` `<link href="<c:url value="/css/common.css"/>" rel="stylesheet" />`
`<link href="`${pageContext.request.contextPath}/css/common.css`" rel="stylesheet"/>`
`// → http://www.ujaen.es/app/css/common.css`

- Si se usan como etiquetas dobles, pueden contener etiquetas para añadir **parámetros adicionales** al queryString de la url:

- `<c:param name="param1 value="valor1">`

35

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Etiquetas Formato

- detalles: <https://jakarta.ee/specifications/tags/3.0/tagdocs/fmt/tld-summary.html>

Tipo	Descripción
<code><fmt:formatNumber></code>	formato de visualización numérico (num decimales, etc.)
<code><fmt:parseNumber></code>	Transforma una cadena (número, moneda, %) a un valor numérico
<code><fmt:formatDate></code>	formatea una fecha según el estilo y patrón indicados
<code><fmt:parseDate></code>	Transforma una cadena (fecha/hora) a un tipo Date
<code><fmt:requestEncoding></code>	Establece la codificación de caracteres de la petición HTTP
<code><fmt:bundle></fmt:bundle></code>	Carga un fichero de identificadores para ser utilizado en su interior
<code><fmt:setBundle></code>	Carga un fichero de identificadores para usarlo en la página. Opcionalmente lo almacena en un ámbito o variable de identificadores especificada (e.g. para reutilizarlo al pasar el control a otro servlet/jsp)
<code><fmt:message></code>	Visualiza el mensaje asociado a un identificador
<code><fmt:setLocale></code>	Establece un identificador de lenguaje (para seleccionar el fichero de idioma)

- Ejemplos:

`<p>Fecha de compra: <fmt:formatDate value="`${producto.compra}`" pattern="dd-MMM-yyyy"></p>`
`<p>Descuento: <fmt:formatNumber value="`${producto.dto}`" maxFractionDigits="2" type="currency"/></p>`

36

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2024



Etiquetas con Funciones auxiliares

- Se suelen utilizar como expresiones en EL. e.g. `${fn:length(nombre)}`
- detalles: <https://jakarta.ee/specifications/tags/3.0/tagdocs/fn/tld-summary.html>

Función	Descripción
<code>fn:contains</code> <code>fn:containsIgnoreCase</code> <code>fn:startsWith</code> <code>fn:endsWith</code>	Comprueba si una cadena contiene una subcadena idem, pero si tener en cuenta mayúsculas/minúsculas Comprueban si una cadena empieza o termina por una subcadena
<code>fn:escapeXml</code>	Cambia caracteres que pueden interpretarse como parte de etiquetas
<code>fn:indexOf</code>	Devuelve la posición donde comienza una subcadena en una cadena
<code>fn:split</code> <code>fn:join</code>	Transforma una cadena en un array de cadenas según un separador Une un array de cadenas utilizando un separador determinado
<code>fn:length</code>	Devuelve el número de elementos en una colección o la long de una cadena
<code>fn:replace</code>	Reemplaza en una cadena las ocurrencias de una subcadena por otra
<code>fn:substring</code> <code>fn:substringAfter</code> <code>fn:substringBefore</code>	Devuelve una subcadena de una cadena Devuelve la cadena restante después de una subcadena Devuelve la cadena previa a una subcadena
<code>fn:toLowerCase</code> <code>fn:toUpperCase</code> <code>fn:trim</code>	Conversión a minúsculas Conversión a mayúsculas Eliminar espacios anteriores y posteriores en una cadena

37

Conclusiones

- La **arquitectura MVC** determina las competencias de los componentes de una aplicación y sus interacciones entre los mismos.
- El uso de **motores de plantillas** para la generación de contenidos html permite separar su contenido (html) de los detalles de programación (java)
- Las **etiquetas JSP** y **Expression Language** son dos tecnologías que permiten utilizar JSP como plantillas para contenidos HTML
- Los **contextos de persistencia en una aplicación web** sirven para el intercambio de datos entre los Servlets y JSP encargados del procesamiento de peticiones HTTP
- **JSTL** es una colección de etiquetas JSP de uso común que simplifica la generación de contenidos en JSP

38

Bibliografía

- Sierra, K, Basham, B. *Head First Servlets and JSP*. 2nd Edition. cap. 3, 8 y 9. O'Reilly Media, 2008. [[Recurso electrónico UJA](#)]
- J. M. Ordax Cassá, Diseño de aplicaciones java EE, en *Programación web en java*. Capítulos 6 y 8. Madrid: Ministerio de Educación de España, 2016. [[Recurso electrónico](#)]

Lecturas Complementarias

- *JavaServer Pages Technology*, in *The Java EE 5 Tutorial*. Oracle, 2010. cap 5. Disponible en <http://docs.oracle.com/javaee/5/tutorial/doc/>
- *JavaServer Pages Standard Tag Library*, in *The Java EE 5 Tutorial*. Oracle, 2010. cap 7. Disponible en <http://docs.oracle.com/javaee/5/tutorial/doc/>
- *Jakarta Server Pages. 3.0*. Eclipse Foundation, 2022. Disponible en <https://jakarta.ee/specifications/pages/3.0/>
- *Jakarta Standard Tag Library. 3.0*. Eclipse Foundation, 2022. Disponible en <https://jakarta.ee/specifications/tags/3.0/>